

```

0000 Vincent MORIN-Universite de Bretagne Occidentale-janvier 2025-Licence CC BY-SA 4.0-1-1-2-3-4-5-6-7-8-9-10-11-12-13-14-15-16-17-18-19-20-21-22-23-24-25-26-27-28-29-30-31-32-33-34-35-36-37-38-39-40-41-42-43-44-45-46-47-48-49-50-51-52-53-54-55-56-57-58-59-60-61-62-63-64-65-66-67-68-69-70-71-72-73-74-75-76-77-78-79-80-81-82-83-84-85-86-87-88-89-90-91-92-93-94-95-96-97-98-99-100-101-102-103-104-105-106-107-108-109-110-111-112-113-114-115-116-117-118-119-120-121-122-123-124-125-126-127-128-129-130-131-132-133-134-135-136-137-138-139-140-141-142-143-144-145-146-147-148-149-150-151-152-153-154-155-156-157-158-159-160-161-162-163-164-165-166-167-168-169-170-171-172-173-174-175-176-177-178-179-180-181-182-183-184-185-186-187-188-189-190-191-192-193-194-195-196-197-198-199-200-201-202-203-204-205-206-207-208-209-210-211-212-213-214-215-216-217-218-219-220-221-222-223-224-225-226-227-228-229-230-231-232-233-234-235-236-237-238-239-240-241-242-243-244-245-246-247-248-249-250-251-252-253-254-255-256-257-258-259-260-261-262-263-264-265-266-267-268-269-270-271-272-273-274-275-276-277-278-279-280-281-282-283-284-285-286-287-288-289-290-291-292-293-294-295-296-297-298-299-300-301-302-303-304-305-306-307-308-309-310-311-312-313-314-315-316-317-318-319-320-321-322-323-324-325-326-327-328-329-330-331-332-333-334-335-336-337-338-339-340-341-342-343-344-345-346-347-348-349-350-351-352-353-354-355-356-357-358-359-360-361-362-363-364-365-366-367-368-369-370-371-372-373-374-375-376-377-378-379-380-381-382-383-384-385-386-387-388-389-390-391-392-393-394-395-396-397-398-399-400-401-402-403-404-405-406-407-408-409-410-411-412-413-414-415-416-417-418-419-420-421-422-423-424-425-426-427-428-429-430-431-432-433-434-435-436-437-438-439-440-441-442-443-444-445-446-447-448-449-450-451-452-453-454-455-456-457-458-459-460-461-462-463-464-465-466-467-468-469-470-471-472-473-474-475-476-477-478-479-480-481-482-483-484-485-486-487-488-489-490-491-492-493-494-495-496-497-498-499-500-501-502-503-504-505-506-507-508-509-510-511-512-513-514-515-516-517-518-519-520-521-522-523-524-525-526-527-528-529-530-531-532-533-534-535-536-537-538-539-540-541-542-543-544-545-546-547-548-549-550-551-552-553-554-555-556-557-558-559-560-561-562-563-564-565-566-567-568-569-570-571-572-573-574-575-576-577-578-579-580-581-582-583-584-585-586-587-588-589-590-591-592-593-594-595-596-597-598-599-600-601-602-603-604-605-606-607-608-609-610-611-612-613-614-615-616-617-618-619-620-621-622-623-624-625-626-627-628-629-630-631-632-633-634-635-636-637-638-639-640-641-642-643-644-645-646-647-648-649-650-651-652-653-654-655-656-657-658-659-660-661-662-663-664-665-666-667-668-669-670-671-672-673-674-675-676-677-678-679-680-681-682-683-684-685-686-687-688-689-690-691-692-693-694-695-696-697-698-699-700-701-702-703-704-705-706-707-708-709-710-711-712-713-714-715-716-717-718-719-720-721-722-723-724-725-726-727-728-729-730-731-732-733-734-735-736-737-738-739-740-741-742-743-744-745-746-747-748-749-750-751-752-753-754-755-756-757-758-759-760-761-762-763-764-765-766-767-768-769-770-771-772-773-774-775-776-777-778-779-780-781-782-783-784-785-786-787-788-789-790-791-792-793-794-795-796-797-798-799-800-801-802-803-804-805-806-807-808-809-810-811-812-813-814-815-816-817-818-819-820-821-822-823-824-825-826-827-828-829-830-831-832-833-834-835-836-837-838-839-840-841-842-843-844-845-846-847-848-849-850-851-852-853-854-855-856-857-858-859-860-861-862-863-864-865-866-867-868-869-870-871-872-873-874-875-876-877-878-879-880-881-882-883-884-885-886-887-888-889-890-891-892-893-894-895-896-897-898-899-900-901-902-903-904-905-906-907-908-909-910-911-912-913-914-915-916-917-918-919-920-921-922-923-924-925-926-927-928-929-930-931-932-933-934-935-936-937-938-939-940-941-942-943-944-945-946-947-948-949-950-951-952-953-954-955-956-957-958-959-960-961-962-963-964-965-966-967-968-969-970-971-972-973-974-975-976-977-978-979-980-981-982-983-984-985-986-987-988-989-990-991-992-993-994-995-996-997-998-999-1000-1001-1002-1003-1004-1005-1006-1007-1008-1009-1010-1011-1012-1013-1014-1015-1016-1017-1018-1019-1020-1021-1022-1023-1024-1025-1026-1027-1028-1029-1030-1031-1032-1033
```

```

--: POSITIVE:= 1;-----| ETAT D ANALYSEUR INITIALISE A 1 AP:= INTEGER; ACTION:= INTEGER; ASYM:= AC_BYTE; NBR_OF_SYLS:= NATUR
AL;-----| NUMBER OF SYLLABLES TO BE-POPPED ZERO_BYTE:= constant AC_BYTE:= 0;-----| procedure DEBUG_PRINT( TXT : STRING )
-----| is begin for I in 2 .. SP loop PUT( " " ); end loop; PUT( 's' & POSITIVE'IMAGE( STATE ) & 's' ); if 2*S
P + TXT'LENGTH > 77 then PUT_LINE( TXT( TXT'FIRST .. 77 - 2 * SP ) ); else PUT_LINE( TXT ); end if; end DEBUG_PRINT;-----
-----| procedure DEBUG_PRINT( V : TREE ) is begin DEBUG_PRINT( PRINT_NAME( V ) ); end DEBUG_PRINT;-----| b
egin LTYPE := LT_END_MARK;-----| INITIALISER A LIGNE VIDE SOURCEPOS := TREE_VOID;-----| INITIALISER LA POSITION SOURCE A VID
E GET_TOKEN;-----| ALLER-CHERCHER UN LEXEME STACK( 1 ).STATE := 1; STACK( 1 ).SRCPOS := SOURCEPOS; STACK( 2 ).SRCPOS := SOUR
CEPOS; SSITOP := 0;-----| START WITH EMPTY-SEMANTIC STACK loop AP := GRMR_TBL.GRMR.AC_SYM( AP ); exit when ASYM = ZERO_BYTE or else
0130 ACTION := AP; else -----| POINTS TO-SHIFT STUFF loop ASYM := GRMR_TBL.GRMR.AC_SYM( AP ); exit when ASYM = ZERO_BYTE or else
ASYM := LEX_TYPE'POS( TOKENSYM ); AP := AP + 1; end loop; ACTION := INTEGER( GRMR_TBL.GRMR.AC_TBL( AP ) ); if AC
TION > 0 then-----| CAN'T BE SEMANTICS SINCE DIDN'T INDIRECT if DEBUG_PARSE then if LTYPE in LT_WITH_SEMANTICS or else LTYPE = L
T_ERROR then DEBUG_PRINT( LEX_IMAGE( LTYPE ) & " " & TOKEN_STRING ); else DEBUG_PRINT( TOKEN_STRING ); end if; if AC
-SEMANTIC STACK IF THIS TOKEN HAS SEMANTICS if LTYPE in LT_WITH_SEMANTICS then if LTYPE = LT_NUMERIC_LIT then AUA := ( TOKEN_ELMT, SOU
RCEPOS, STORE_TEXT( TOKEN_STRING ) ); else AUA := ( TOKEN_ELMT, SOURCEPOS, STORE_SYM( TOKEN_STRING ) ); end if; if
if LTYPE = LT_END_MARK then-----| ARRIVE EN FIN DE COMPILE if SP = 2 then PUT_LINE( "FIN COMPILE MAIS SP = " & INTEGER'IM
AGE( SP ) ); end if; if SSITOP = 1 then PUT( "FIN COMPILE MAIS SSITOP = " & INTEGER'IMAGE( SSITOP ) ); else AUA := SEMSTAK( 1 ); if A
UA.KIND /= NODE_ELMT then PUT_LINE( "FIN COMPILE MAIS SEMSTAK(1) PAS UN NOEUD. " ); else-----| SAUVER L'ARBRE SYNTAXIQUE-DANS LE XD_ST
RUCTURE DU USER_ROOT D( XD_STRUCTURE, USER_ROOT, AUA.ELMT ); if DEBUG_PARSE then PRINT_NODE( D( XD_STRUCTURE, USER_ROOT ) ); end if; e
0140 nd if; end if; exit; end if; SP := SP + 1; STATE := ACTION; STACK( SP ).STATE := ACTION; STACK( SP ).SRC
POS := SOURCEPOS; STACK( SP+1 ).SRCPOS := SOURCEPOS; GET_TOKEN; else if ACTION = 0 then-----| ERREUR DE SYNTAXE
ERROR( SOURCEPOS, "ERREUR DE SYNTAXE - " & SLINE.BDY( F_COL..E_COL ) );-----| INSERE UN NOEUD-ERREUR ET-AFFICHE UN MESSAGE exit; el
se-----| SEMANTIC AND REDUCE ACTIONS NODE_CREATED := FALSE; loop if ACTION > -10000 then -----| TRANSFER TO SEMANTIC ACTION TABLE
AP := --ACTION; -- TRANSFER-IN TABLE loop ACTION := INTEGER( GRMR_TBL.GRMR.AC_TBL( AP ) ); exit when ACTION <= 0; BUILD_TREE( ACT
ION, AP );-----| CONSTRUCTION DE-L'ARBRE INCREMENTE AP EN INTERNE end loop; end if; if ACTION > -30000 and ACTION <= -10000 then
--REDUCE ACTION := --ACTION - 10000; NBR_OF_SYLS := ACTION/1000; ACTION := ACTION mod 1000; -- I.E., RULE if SP = SP - NBR_OF_SYL
S; -- POP THE STACK STATE := STACK( SP ).STATE; SEMSTAK( SSITOP ).SPOS := STACK( SP+1 ).SRCPOS; if NODE_CREATED and then SEMSTAK( SSITOP ).K
IND = NODE_ELMT and then SEMSTAK( SSITOP ).ELMT /= TREE_VOID then D( LX_SRCPOS, SEMSTAK( SSITOP ).ELMT, SEMSTAK( SSITOP ).SPOS ); end if; e
d if; -- FIND GOTO FOR NONTERMINAL IN-THIS STATE AP := GRMR_TBL.GRMR.ST_TBL( STATE ); loop AP := AP - 1; ASYM := GRMR_TBL.GRMR
0150 R.AC_SYM( AP ); if ASYM = ZERO_BYTE then PUT_LINE( "!! ***** NONTER GOTO NOT FOUND. " ); raise PROGRAM_ERROR; end if;
exit when INTEGER( ASYM ) = ACTION; end loop; STATE := INTEGER( GRMR_TBL.GRMR.AC_TBL( AP ) ); SP := SP + 1; STACK( SP ).STATE := STATE;
if NBR_OF_SYLS = 0 then-----| NULLABLE REDUCTION; SRCPOS-NOT ALREADY THERE STACK( SP ).SRCPOS := SOURCEPOS; end if; STACK( SP+1 ).SRCPOS
:= SOURCEPOS; exit; else PUT_LINE( "!! PARSE_TABLE_ERROR " ); raise PROGRAM_ERROR; end if; end loop; end if; end loop;
end PARSE_COMPILATION;-----| begin READ_PARSE_TABLES;-----| TABLES DE LA GRAMMAIRE LUES DANS PARSE.BIN OPEN( IFILE, IN_FIL
E, PATH_TEXTE & NOM_TEXTE );-----| OUVRIER LE FICHER SOURCE A COMPILER PUT( "ada83 compiling " & PATH_TEXTE & NOM_TEXTE ); CREATE_IDL_TREE_FI
LE( IDL.LIB_PATH( 1..LIB_PATH_LENGTH ) & "$$.TMP " );-----| CREER-LE FICHER D'ARBRE (AVEC SON NOEUD RACINE DE TYPE-DN_ROOT) USER_ROOT := MAKE( DN
_USER_ROOT );-----| CREER-UN NOEUD RACINE SECONDAIRE DU-TYPE DN_USER_ROOT D( XD_USER_ROOT, TREE_ROOT, USER_ROOT );-----| NOEUD-USER_ROOT-
DANS LE CHAMP XD_USER_ROOT DU-NOEUD TREE_ROOT D( XD_SOURCENAME, USER_ROOT, STORE_TEXT( NOM_TEXTE ) );-----| NOM DU SOURCE DANS LE CHAMP XD_XOURC
ENAME DU NOEUD USER_ROOT PARSE_COMPILATION;-----| EFFECTUER LA PHASE D'ANALYSE SYNTAXIQUE DU SOURCE D( XD_SOURCE_LIST, TREE_ROOT, SOURCE
LIST.FIRST );-----| STOCKE LA LISTE-SOURCE_LIST DANS L'ATTRIBUT XD_SOURCE_LIST CLOSE( IFILE ); CLOSE_PAGE_MANAGER;-----| FERMER LE FICH
IER ARBRE exception when NAME_ERROR => PUT_LINE( "FILE NOT FOUND : " & PATH_TEXTE & NOM_TEXTE ); raise; end PAR_PHASE;-----| --1-2-
3-4-5-6-7-8-9-10-11-12

```